Choosing Versus Accepting: Why Cheaper Code Is Producing Costlier Debt

Ook Lee, *Hanyang University, Seoul, 04763, South Korea*

Abstract*—Large language model (LLM) coding assistants have made working code cheap to produce, yet early data indicate rising duplication and short-lived churn alongside declining refactoring. The conventional view of technical debt as a deliberate trade-off cannot explain why debt should grow just as sound code becomes cheaper. We argue that assistants change less the price of quality than the conditions under which developers decide whether to pursue it, engaging present bias, the fluency heuristic, automation bias, default effects, loss aversion, weakened ownership, and misaligned incentives. Debt that teams once chose is increasingly debt they accept without noticing. We describe observable indicators of each mechanism, report a within-developer comparison of 32,620 commits in five open-source projects that partly supports the account, and propose inexpensive changes to the development environment that target each mechanism.

Actionable insights for practitioners

- ➔ *Treat a working suggestion as a draft*—A fluent, passing AI suggestion still needs a design decision, and review depth should follow the size of the change rather than the tool's track record.
- ➔ *Measure what assistants make easy to hide*—Report net lines, self-admitted debt markers, and refactoring share for AI-assisted changes alongside throughput.
- ➔ *Change defaults before adding policies*—Diff-based acceptance, owner sign-off for cross-module changes, and team-level maintainability metrics act on the biases directly.

The speed with which LLM-based coding assistants have been adopted has few precedents among development tools, and industry surveys now report that most professional developers use them daily. In terms of output, the tools largely deliver what they promise. The evidence on what becomes of the code after it has been merged is less reassuring.

In an analysis of more than 200 million changed lines, GitClear found that copy-pasted lines overtook moved, or refactored, lines for the first time in its data as assistant use spread, while duplicated blocks rose several-fold and more new code was revised within two weeks of being committed.[1] A study of more than 300,000 AI-authored commits in 6,299 GitHub repositories found that code smells made up

XXXX-XXXX © 2021
Digital Object Identifier 10.1109/XXX.0000.000000

89.3% of the issues these commits introduced and that 22.7% of the tracked issues remained in the latest revision.[2]

These findings sit uneasily with the usual account of technical debt. Since Cunningham introduced the metaphor,[3] debt has been framed as a reasoned trade-off in which a team knowingly takes a shortcut to deliver sooner and accepts the "interest" of later rework.[4] On this account, debt should decline when quality becomes cheaper, and an assistant can generate a well-structured, tested module in roughly the time it takes to generate a careless one.

We suggest that the explanation lies in the circumstances in which developers make quality decisions rather than in the cost of those decisions. Behavioral economics has documented how such circumstances shape judgment, and LLM assistants alter them in ways that engage several known biases at once.

RELATION TO PRIOR WORK

The role of judgment in technical debt has been examined before. Interviews with software architects have shown that biases such as anchoring and optimism give rise to architectural debt, although that work predates LLM assistance.[5] An observational study of developers working with LLMs classified 48.8% of their actions as biased, with interactions involving the model accounting for 56.4% of the biased actions.[6] Other work has traced the issues introduced by AI-authored commits at scale[2] and simulated the long-run effects of AI assistance on software quality.[7]

Our account differs in three respects. It asks why debt should rise at all once quality has become cheaper, and it locates the answer in the order in which options reach the developer rather than in gaps of knowledge or effort. It distinguishes debt that is chosen from debt that is accepted, a distinction that bears on whether debt is ever recorded. It also ties each mechanism to an indicator that can be computed from version-control data and to an intervention that addresses it.

DEBT THAT IS CHOSEN AND DEBT THAT IS ACCEPTED

Debt can be viewed as the cumulative result of many small decisions in which a developer either adopts an expedient implementation or invests in a sound one. The rational view holds that the expedient option is taken when the time it saves outweighs the rework it will require, and it therefore expects cheaper quality to be chosen more often.

What this view overlooks is that assistants also change the order in which the options are encountered. Without an assistant, both options lie in the future and are weighed before any code is written. With one, a plausible implementation is already on the screen when the quality question arises, so the decision becomes whether to revise something that appears to work. People facing a decision of this kind tend to keep what is in front of them.

The resulting debt is of a particular kind. Inadvertent debt has usually been attributed to gaps in knowledge,[4] whereas here the developer could recognize the shortcut but meets it in a form that does not invite scrutiny. Such debt is accepted rather than chosen, and it often goes unrecorded. Developers who knowingly cut corners tend to leave self-admitted debt markers such as TODO or HACK comments,[8] but code that looks finished seldom attracts them.

HOW ASSISTANTS ENGAGE FAMILIAR BIASES

None of the mechanisms discussed here is new. Our argument is that LLM assistance engages them jointly, at the level of the individual developer, the code review, and the team (see Table 1).

TABLE 1. Behavioral mechanisms engaged by LLM assistance.

Level	Mechanism	How the assistant engages it	Expected signal in the data
Individual	Present bias[9]	Working code is available at once; improving it is a later, effortful step	Declining refactoring share; deferred clean-up comments left unresolved
Individual	Fluency heuristic[10]	Generated code is consistently formatted, well named, and fluently explained	Fewer self-admitted debt markers relative to statically detected debt
Individual	Loss aversion[11]	Deleting generated code feels like discarding something of value	More added than deleted lines; growth in duplicated and dead code
Individual	Weakened ownership[12]	Little personal effort is invested in code obtained from a prompt	Slower response to defects in generated code
Review	Automation bias[13]	Reviewers calibrate scrutiny to the tool's general reliability	Less review time per changed line on AI-assisted pull requests
Review	Default effect[14]	Accepting a suggestion takes one keystroke; rejecting it takes more	Acceptance rates that track interface cost more than suggestion quality
Team	Moral hazard and the commons[15]	Developers can modify code they do not own, while others bear the upkeep	Debt concentrated in modules changed by nonowners; duplication rising with team size
Team	Goodhart's law and rebound	Throughput metrics become easy to inflate as code becomes cheap	Faster debt growth in teams evaluated on individual throughput

The Individual Developer

Present bias is the tendency to weight immediate rewards more heavily than any constant discount rate would imply, and its symptom is the plan that is repeatedly postponed.[9] When an assistant returns working code, the developer weighs a reward available now against an investment that pays off later, which is where present bias is strongest. The intention to tidy the code before release tends to yield to the next task.

The fluency heuristic leads people to judge material that is easy to process as more accurate.[10] A hurried human shortcut usually looks like one, being terse and unexplained, whereas a generated shortcut is neatly formatted and accompanied by a confident explanation. The cues that would prompt a second look are largely absent.

Loss aversion and status quo bias[11] suggest that once generated code is in the working tree, removing it feels like a loss, even when it duplicates an existing utility. Because the code cost almost no effort, the sense of sunk cost that would prompt the question of whether it is needed is also missing. Research on the IKEA effect, finally, shows that people value what they have labored over,[12] so code obtained from a prompt is likely to attract weaker ownership, less documentation, and slower attention when it fails.

The Code Review

Code review is the main safeguard against individual lapses, but automation bias leads people to verify automated output less carefully when the aid is usually correct.[13] Reviewers may calibrate scrutiny to the tool's general reliability rather than to the change in front of them, and larger, more numerous generated pull requests leave less attention per line. Interface design adds to this. An inline suggestion is accepted with one keystroke while rejecting it takes more, and preselected options are chosen far more often than preferences alone would predict.[14] A developer who accepts a suggestion has often not so much chosen it as failed to decline it.

The Team

Assistants make it easier to change code one does not fully understand, including modules owned by others, so the author gains the delivery credit while the owner inherits the upkeep, a close analogue of moral hazard. A

codebase is also a shared resource whose coherence every contributor draws on,[15] and when generating a new utility is cheaper than finding an existing one, duplication accumulates. Throughput metrics reinforce these dynamics. Once assistants make merged pull requests cheap to increase, such metrics become targets that can be met without delivering the value they were meant to reflect, as Goodhart's law predicts.

RECOGNIZING THE PATTERN IN A REPOSITORY

Each mechanism leaves traces in data that most teams already collect. Where assistant use can be identified from co-author trailers, agent-opened pull requests, or telemetry, AI-assisted and other changes can be compared, ideally for the same developers over the same period. Six indicators are useful:

- *Refactoring share*—the proportion of changed lines that are moved or restructured rather than added.
- *Recorded versus detected debt*—the ratio of self-admitted debt markers to issues flagged by static analysis.
- *Net additions*—the ratio of added to deleted lines, together with duplicated and unreachable code.
- *Review depth*—review time and substantive comments per changed line.
- *Ownership of change*—the share of AI-assisted changes made by developers other than a module's principal maintainer.
- *Defect response*—the time taken to address defects traced to generated code.

No single indicator isolates one mechanism. Rising duplication, for instance, fits both loss aversion and commons dynamics, but duplication that grows with the number of contributors points toward the commons, whereas a net-addition bias in single-owner modules points toward loss aversion.

A FIRST LOOK AT OPEN-SOURCE PROJECTS

To ground the account empirically, we examined three of these indicators in six actively maintained open-source projects with many contributors: Prefect, Cal.com, n8n, Grafana, Home Assistant, and Sentry. A commit was treated as AI-assisted when its message carried a co-author trailer or footer naming a coding assistant such as Claude, Codex, Cursor, or Copilot. We analyzed nonmerge commits from April 2025 to October 2026, excluding bot accounts, nonsource files, generated code, and commits that added more than 5000 lines.

Because developers differ in style and tasks, we compared each developer with themselves. For every developer with at least five AI-labeled commits, we compared those commits with the same developer's unlabeled commits made after the first labeled one, retaining developers with at least five of each. This left 251 developers in five projects (Prefect had too few) and 32,620 commits, of which 10,641 carried an AI label (see Table 2).

TABLE 2. Within-developer comparison of AI-labeled and other commits (251 developers, five projects).

Indicator	AI-labeled commits	Other commits	Developers for whom AI-labeled commits score higher	*p*
Additions as share of changed lines, median across developers	82.5%	78.8%	61%	.002[a]
Self-admitted debt markers per 1000 added lines, all commits pooled	0.16	0.20	22% (29% lower, 49% tied)	.16[b]

Indicator	AI-labeled commits	Other commits	Developers for whom AI-labeled commits score higher	*p*
Added lines duplicated within the same commit, median across developers	32.3%	33.0%	49%	.93[a]
Changed lines per commit, median across developers	152	85	72%	<.001[a]

[a]Wilcoxon signed-rank test across developers.

[b]Sign test excluding ties.

The results partly support the account. AI-labeled commits were more weighted toward additions for 61% of developers, which is consistent with the net accumulation that loss aversion predicts. They carried about 20% fewer self-admitted debt markers per line, which is consistent with debt that is accepted without being recorded, although the difference was not statistically reliable. Duplication showed no difference, so within-developer comparisons do not reproduce the rise reported in aggregate industry data.

Two qualifications apply. AI-labeled commits were considerably larger, and among commits adding 10 to 300 lines the difference in additions narrowed and lost statistical reliability (57% of 170 developers, $p = .23$). Moreover, the labels capture only disclosed assistance, and some unlabeled commits were almost certainly AI-assisted, which biases all three comparisons toward zero. We therefore treat these figures as a plausibility check rather than a test.

REDESIGNING THE DECISION ENVIRONMENT

Standard remedies such as debt registers and repayment sprints presuppose that debt is visible and will be repaid once the figures justify it. If much of it is never recorded and repayment is continually deferred, they will fall short. Choice architecture, the deliberate design of the setting in which decisions are made,[14] suggests less expensive measures aimed at specific mechanisms:

- *Review defaults*—Route AI-assisted changes to a review track with a minimum expected time per changed line, and ask reviewers to name any existing utility the change could have reused.
- *Deletion prompts*—Report net rather than added lines in pull request summaries, and flag unreferenced functions at commit time, so that retaining code requires justification.
- *Ownership rules*—Require owner approval for changes that cross module boundaries, and assign defects to the author of the originating change for a defined period.
- *Interface friction*—Present large generated blocks as diffs to be reviewed rather than as text to be accepted in place.
- *Balanced metrics*—Pair every throughput metric with a maintainability metric of equal frequency and visibility, and evaluate teams rather than individuals.

Most of these measures can be introduced within existing tools, and because each targets one mechanism, a team can adopt one and observe whether the corresponding indicator responds.

LIMITATIONS

The account is a theoretical explanation, and the analysis above is a plausibility check rather than a test. Like most available evidence, it draws on public repositories, while the industrial settings in which most commercial software is built remain underexamined. The mechanisms in Table 1 should therefore be read as hypotheses, and replication in industry would be especially valuable. A fuller test would combine repository mining with a controlled review experiment that varies stated authorship independently of actual

authorship, and with a field study of organizations that differ in how they measure and reward developers.

AI-authorship labels are incomplete, and static analysis only approximates the debt developers experience. The argument also depends on the present gap between how finished generated code appears and how sound it is. Should that gap narrow, the fluency mechanism would weaken, although the incentive-related mechanisms would likely persist. Nor do we claim that generated code is inherently inferior, only that the conditions under which it is produced and reviewed make its weaknesses more likely to be accepted unexamined.

CONCLUSION

LLM assistants have made code inexpensive to produce, and the early evidence suggests that they may be making systems more expensive to maintain. We have argued that this reflects the conditions under which quality decisions are now made rather than any change in their underlying economics. A working artifact arrives before the quality question is posed, shortcuts look like finished work, acceptance is the default, deletion feels like a loss, and those who choose are often not those who pay. If the account is broadly correct, the most promising remedies lie in the design of the environment in which debt is incurred, and the assistants themselves are a natural place to build that design in.

ACKNOWLEDGMENTS

The author used an AI assistant (Claude, Anthropic) for language editing, for restructuring the manuscript, and for writing the commit-mining scripts. The author reviewed all output and takes full responsibility for the content.

REFERENCES

1. GitClear, "AI copilot code quality: 2025 data suggests 4x growth in code clones," research report, 2025. [Online]. Available: https://www.gitclear.com/ai_assistant_code_quality_2025_research
2. Y. Liu, R. Widyasari, Y. Zhao, I. C. Irsan, J. Chen, and D. Lo, "Debt behind the AI boom: A large-scale empirical study of AI-generated code in the wild," 2026, *arXiv:2603.28592*.
3. W. Cunningham, "The WyCash portfolio management system," in *Addendum Proc. OOPSLA*, 1992, pp. 29–30.
4. P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Softw.*, vol. 29, no. 6, pp. 18–21, 2012.
5. K. Borowa, A. Zalewski, and S. Kijas, "The influence of cognitive biases on architectural technical debt," in *Proc. IEEE 18th Int. Conf. Softw. Archit. (ICSA)*, 2021, doi: 10.1109/ICSA51549.2021.00019.
6. X. Zhou, Z. Saghi, S. Sabouri, R. Pandita, M. McGuire, and S. Chattopadhyay, "Cognitive biases in LLM-assisted software development," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, doi: 10.1145/3744916.3773104.
7. E. G. Anderson, G. G. Parker, and B. Tan, "Generative AI and software quality dynamics," in *Proc. Int. Syst. Dyn. Conf.*, 2026.
8. A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *Proc. IEEE Int. Conf. Softw. Maint. Evol. (ICSME)*, 2014, pp. 91–100.
9. T. O'Donoghue and M. Rabin, "Doing it now or later," *Amer. Econ. Rev.*, vol. 89, no. 1, pp. 103–124, 1999.
10. A. L. Alter and D. M. Oppenheimer, "Uniting the tribes of fluency to form a metacognitive nation," *Pers. Soc. Psychol. Rev.*, vol. 13, no. 3, pp. 219–235, 2009.
11. D. Kahneman and A. Tversky, "Prospect theory: An analysis of decision under risk," *Econometrica*, vol. 47, no. 2, pp. 263–291, 1979.
12. M. I. Norton, D. Mochon, and D. Ariely, "The IKEA effect: When labor leads to love," *J. Consum. Psychol.*, vol. 22, no. 3, pp. 453–460, 2012.
13. R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010.
14. R. H. Thaler and C. R. Sunstein, *Nudge: Improving Decisions About Health, Wealth, and Happiness*. New Haven, CT, USA: Yale Univ. Press, 2008.
15. E. Ostrom, *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge, U.K.: Cambridge Univ. Press, 1990.

OOK LEE is a [position] in the [department] at Hanyang University, Seoul, 04763, South Korea. His research interests include [interest 1], [interest 2], and [interest 3]. Lee received the [degree] in [field] from [institution]. He is a member of [IEEE / IEEE Computer Society]. Contact him at ooklee@hanyang.ac.kr.